

PORTADA

Arquitectura de Software

Semana 12 — Desarrollo basado en Componentes

Reutilizar software sin pagar el precio en secreto · Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

Dos tercios de internet. Una línea de código.

Heartbleed, CVE-2014-0160 — el defecto en OpenSSL que estuvo dos años sin descubrirse en el componente de cifrado más usado del planeta.



TRANSICIÓN

La Unidad 3 empieza por lo que no se ve

La Unidad 2 les enseñó a decidir la estructura de un sistema: patrones, estilos, principios. La Unidad 3 — **Desarrollo de Software basado en Arquitectura** — arranca hoy con una pregunta distinta: ¿de qué está hecho realmente ese sistema por dentro?

La respuesta casi siempre incluye piezas que su equipo no escribió: componentes de terceros, librerías, servicios reutilizados. Heartbleed muestra qué pasa cuando una de esas piezas falla.

CASO · FICHA TÉCNICA

Heartbleed, OpenSSL, 2014

Identificador	CVE-2014-0160
Componente afectado	OpenSSL — implementación de la extensión "heartbeat" del protocolo TLS
Bug introducido	14 de marzo de 2012, en OpenSSL 1.0.1, por un contribuidor voluntario (Robin Seggelmann)
Tiempo sin detectar	Aproximadamente 2 años
Hecho público	7 de abril de 2014
Descubridores	Neel Mehta (investigador de seguridad de Google) y la firma finlandesa Codenomicon, de forma independiente

Uno de los casos más citados en la industria para hablar del riesgo de depender de componentes compartidos.

CASO · QUÉ PASABA TÉCNICAMENTE

Un "heartbeat" que respondía de más

La extensión heartbeat de TLS permite que un cliente envíe un mensaje corto y el servidor lo devuelva igual, para confirmar que la conexión sigue viva. El bug estaba en cómo OpenSSL manejaba la longitud declarada de ese mensaje.

- El cliente podía declarar una longitud de respuesta **mayor a la del mensaje real que envió**, sin que el servidor la validara correctamente.
- OpenSSL respondía copiando esa cantidad de memoria del servidor, aunque el mensaje original fuera mucho más corto.
- Resultado: un atacante podía leer **hasta 64 KB de memoria del servidor por cada solicitud** — repetible cuantas veces quisiera.

CASO · QUÉ SE FILTRABA

Sin dejar rastro en ningún log

Esos fragmentos de memoria del servidor podían contener **claves privadas, contraseñas y tokens de sesión** — exactamente lo que un servidor TLS necesita mantener en memoria para operar. Y como el heartbeat es tráfico normal del protocolo, **el ataque no dejaba rastro en los registros del servidor.**

No era necesario romper el cifrado: bastaba con leer lo que el propio servidor tenía en memoria en ese instante.

CASO · LA ESCALA DEL PROBLEMA

Un componente en dos tercios de internet

OpenSSL era, en ese momento, el software TLS más desplegado en servidores web — estimaciones de Netcraft lo situaban en **aproximadamente dos tercios de los sitios web activos de la época**. Se estimó que varios cientos de miles de servidores eran vulnerables.

Y sin embargo, el proyecto OpenSSL —embebido en gran parte de la infraestructura crítica de internet— era mantenido mayormente por voluntarios, con un financiamiento del orden de unos pocos miles de dólares al año.

CASO · CONSECUENCIA INSTITUCIONAL

Cuando el problema es de financiamiento, no solo de código

Heartbleed dejó una lección incómoda: la infraestructura de seguridad de gran parte de internet dependía de un componente que casi nadie auditaba ni financiaba de forma sostenida.

Esto impulsó la creación de la Core Infrastructure Initiative de la Linux Foundation, un fondo específico para financiar el mantenimiento de proyectos de código abierto que resultan críticos para la infraestructura de internet, precisamente para que un componente tan usado como OpenSSL no vuelva a depender de la buena voluntad de unos pocos.

TRANSICIÓN

El problema no es reutilizar

OpenSSL fue reutilizado por millones de sistemas precisamente porque reutilizar es más eficiente que reescribir criptografía TLS desde cero en cada proyecto. Ese es el argumento central del **desarrollo basado en componentes**.

El problema de Heartbleed no fue reutilizar un componente — fue integrarlo ampliamente sin gestionar activamente la confianza, la versión y el mantenimiento de esa pieza compartida. Para entender por qué, hay que empezar por definir qué es exactamente un componente.

CONCEPTO

¿Qué es un componente de software?

Un componente es una unidad de software desplegable de forma independiente, que encapsula su implementación interna y expone su funcionalidad únicamente a través de interfaces bien definidas.

- **Interfaz provista:** lo que el componente ofrece a quien lo usa (los métodos, servicios u operaciones que expone).
- **Interfaz requerida:** lo que el componente necesita de otros componentes para funcionar (sus dependencias declaradas).
- **Reemplazable:** mientras un componente nuevo cumpla la misma interfaz, puede sustituir al anterior sin romper el resto del sistema.

OpenSSL es exactamente esto: un componente con una interfaz de funciones criptográficas, integrado en miles de sistemas distintos sin que cada uno conociera su implementación interna.

CONCEPTO · DIFERENCIA CLAVE

Componente no es lo mismo que objeto o clase

	Objeto / Clase	Componente
Granularidad	Una unidad de código, dentro de un mismo programa compilado	Puede contener muchas clases; se empaqueta como una sola unidad
Despliegue	Se compila junto con el resto del sistema	Se despliega como unidad binaria independiente (librería, paquete, servicio)
Conocimiento necesario	Requiere entender la clase para extenderla (herencia)	Solo requiere conocer su interfaz , nunca su implementación interna

Ningún equipo que usó OpenSSL necesitaba leer su código fuente para integrarlo — solo necesitaba llamar a su interfaz. Esa comodidad es, a la vez, la base de la eficiencia de CBSE y el origen de su riesgo.

CONCEPTO

CBSE: Desarrollo basado en Componentes

Component-Based Software Engineering (CBSE) es el enfoque de construir sistemas ensamblando componentes ya existentes en lugar de escribir toda la funcionalidad desde cero.

- **Reutilización:** una misma pieza de software (criptografía, autenticación, logging) se usa en muchos sistemas distintos.
- **Reducción del tiempo de desarrollo:** integrar un componente probado es más rápido que reescribir su funcionalidad.
- **Especialización de equipos:** cada equipo se concentra en su dominio, en vez de dominar criptografía, bases de datos y UI a la vez.
- **Reemplazo independiente:** un componente se puede actualizar o sustituir sin recompilar todo el sistema, si respeta la misma interfaz.

CONCEPTO · EL RIESGO QUE HEARTBLEED HIZO VISIBLE

El "component trust problem"

El problema de confianza en componentes: usar un componente exige confiar en su corrección y seguridad sin haber auditado —ni tener siempre la posibilidad práctica de auditar— su código fuente completo.

Cuanto más sistemas integran el mismo componente, más se amplifica cualquier defecto que contenga. Con OpenSSL, miles de organizaciones confiaron en un componente que casi nadie —ni siquiera sus propios usuarios más grandes— había revisado línea por línea.

Reutilizar reduce el trabajo de escribir código, pero no elimina la responsabilidad de gestionar qué se está confiando y en qué versión.

CONCEPTO · OTRO RIESGO DE CBSE

El "component mismatch problem"

El problema de desajuste de componentes: un componente probado y correcto en el contexto para el que fue diseñado puede fallar al integrarse en un contexto distinto, con supuestos diferentes sobre unidades, formatos de datos o condiciones de operación.

Conexión con lo ya visto (Semana 5): Ariane 5 no fue un caso de componente defectuoso en sí — fue software del Ariane 4, correcto en su contexto original, reutilizado sin ajustar en un cohete con parámetros de vuelo distintos.

Trust problem pregunta "¿puedo confiar en lo que hay dentro?"; mismatch problem pregunta "¿este componente asume el mismo contexto que mi sistema?" — son dos riesgos distintos de CBSE.

CONCEPTO

COTS: componentes listos para usar

COTS (Commercial Off-The-Shelf) son componentes comerciales, ya contruidos y probados por un tercero, que se adquieren o licencian listos para integrar — en contraste con desarrollar el componente propio o adoptar una alternativa de código abierto.

	COTS comercial	Open source	Desarrollo propio
Costo inicial	Licencia	Gratuito (en general)	Alto (tiempo de equipo)
Soporte	Contractual, con proveedor	Comunidad, variable	Interno
Auditoría del código	Casi nunca posible	Posible, rara vez ejercida	Total

OpenSSL es open source: el código era técnicamente auditable por cualquiera — el fallo no fue de acceso al código, sino de que casi nadie invertía tiempo ni dinero en auditarlo o mantenerlo activamente.

CONCEPTO · CÓMO SE EMPAQUETAN HOY

Modelos de componentes

Un **modelo de componentes** define las reglas técnicas concretas —formato de empaquetado, forma de declarar interfaces, mecanismo de versionado— con las que un ecosistema de desarrollo distribuye e integra componentes.

Modelo	Ecosistema	Unidad de empaquetado
EJB (Enterprise JavaBeans)	Java empresarial	Componentes desplegados en un servidor de aplicaciones
Assemblies / NuGet	.NET	Paquetes versionados con dependencias declaradas
Paquetes npm	JavaScript / Node.js	Módulos publicados en un registro público, con versión semántica

OpenSSL no se distribuye por ninguno de estos registros gestionados: se compila desde el código fuente o se empaqueta por cada sistema operativo — otra razón por la que actualizar la versión vulnerable, tras el 7 de abril de 2014, tomó tiempo en propagarse.

TENSIÓN CONCEPTUAL

Reutilizar no es gratis

La promesa de CBSE es real: nadie debería reescribir TLS, un ORM o un framework de UI desde cero para cada proyecto. Pero cada componente externo que se integra es también **una dependencia que alguien más mantiene, versiona y —a veces— deja de mantener.**

El trade-off central: cada componente reutilizado ahorra tiempo de desarrollo hoy, a cambio de heredar el riesgo de seguridad, la calidad y el ciclo de vida de mantenimiento de un tercero mañana. Ese costo no aparece en el cronograma del proyecto — aparece años después, como en Heartbleed.



INTERACCIÓN

Auditen las dependencias de su propio proyecto

En equipos, revisen su proyecto de CodeF@ctory y respondan:

1. ¿Qué componentes o librerías externas (paquetes npm, NuGet, o equivalentes) usa su sistema, que su equipo no escribió?
2. De esos, ¿cuáles son COTS/comerciales y cuáles open source? ¿Alguien del equipo ha revisado su código o solo confían en que funciona?
3. Si uno de esos componentes tuviera una vulnerabilidad crítica mañana, ¿cómo se enterarían y qué tan rápido podrían reemplazarlo?

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. Un **componente de software** es una unidad desplegable de forma independiente, con interfaces provistas y requeridas bien definidas, reemplazable sin conocer su implementación interna.
2. **CBSE** reutiliza componentes para ganar velocidad de desarrollo y permitir especialización de equipos, a cambio de asumir riesgos que hay que gestionar activamente.
3. El **component trust problem** (confiar sin auditar) y el **component mismatch problem** (fallar por contexto distinto, como en Ariane 5) son los dos riesgos centrales de CBSE.
4. **COTS** son componentes comerciales listos para usar; se distinguen de componentes open source o de desarrollo propio por su nivel de auditoría posible y su modelo de soporte.
5. **Heartbleed** (OpenSSL, CVE-2014-0160, 2014) mostró el costo real del trust problem: un bug de 2012, sin descubrir por 2 años, en un componente presente en dos tercios de internet, mantenido con recursos mínimos hasta que se creó la Core Infrastructure Initiative.

CIERRE

Para la próxima semana

- Hacer el inventario de dependencias externas de su proyecto de CodeF@ctory que empezaron a construir en la actividad de hoy.
- Repasar la diferencia entre component trust problem y component mismatch problem — sabrán identificar cuál aplica a cada riesgo concreto de un componente.

El jueves de la Semana 12 seguimos dentro de la Unidad 3 con Proceso de Desarrollo orientado a la Arquitectura: cómo la arquitectura deja de ser solo una decisión de diseño y empieza a guiar el proceso de desarrollo completo.

